CEN 4020 | AI PROJECT · PART 4

EventHub

Software Implementation and AI Development Report

Team 5 · Jaideep Yarasingu (Gold member) · October 25, 2026

Review requested from John Bachta before team submission. The package is a new Part 4 working draft and does not overwrite or push changes to the existing Part 3 repository.

What this report contains

Section	What it covers	Grade weight
1. Changes after Call 4	What the discussion changed in design and implementation, and why	20%
2. AI development process	Inputs, code creation, corrections, prompt improvements and review	60%
3. Testing and evidence	Unit tests, behavior verified, limitations and CI setup	15%
4. Source code and handoff	File structure, commands, and reviewer checklist	5%

Result at the time of drafting: 33 locally executable tests passed, with 2 test files skipped because Flask and Werkzeug could not be installed in this environment. The copied Part 3 eligibility checker returned 0 failures (its optional Structurizr CLI check was unavailable). The report makes no claim that the full integration suite or the actual team CI already passed.

Scope note: Call 4 says some questions were answered in customer emails, but those emails were not part of the provided documents. Those rules remain openly documented as pending rather than guessed.

1. What changed after Call 4

Parts 1–3 gave us the feature requirements and the top-level design. Part 4 is where we had to turn that design into code that can be tested. I kept the Part 3 folders, requirements, decision records and C4 diagrams unchanged. I added independent Python modules and tests beside them so John can compare the implementation against our earlier decisions without losing that work.

1.1 Changes from the implementation discussion

Call 4 decision / concern	What I implemented and why
Build in dependency order	Identity and catalog services first, then pricing, inventory and checkout. Each module can be imported into a test directly; the Flask routes connect them.
Human review and automatic checks	Added CONTRIBUTING.md, an opt-in pre-commit hook, and GitHub Actions workflow. The hook warns if application code changes without an updated specification/alignment note.
One place for pricing	A single compute_quote() function handles base price, quantity, discount and currency. Templates only display the result.
Normalize email before discount checks	Casefold and trim email text. Uppercase @USF.EDU qualifies; plus-address and subdomain policy remain unconfirmed.
Log purchases within the flow	Successful order, inventory decrement and audit are committed together. On a failed attempt the transaction is rolled back and one failure event is recorded.
Event list interactions	A click opens an event page. Double-click requests a one-ticket simulated purchase. This is browser behavior, not a real payment.
Definition of finished	Documented the review, passing tests, specification alignment and CI conditions that must all hold before a feature should be merged.

1.2 Decisions still requiring the customer emails

Call 4 says the ticket hold, discount scope, confirmation rule and purchase-limit arguments were resolved through email, but the transcript does not state the resulting choices. Those emails were not provided. I therefore preserved the earlier no-hold prototype behavior, kept purchase caps configurable and used the on-screen simulated confirmation from Part 3. They are review items, not verified final business decisions.

The discount percentage is explicitly still missing in Call 4. For a USF email address, the prototype will stop checkout with a clear configuration message until the percentage is supplied. This is safer than quietly choosing 10% or another number. Currency examples, exchange rates, rounding order and the 30-minute idle timeout are labeled demo assumptions in the README.

1.3 Changes to the system model

The Part 3 C4 structure still applies: Buyer and Administrator are the two people; the containers are one Flask application, a SQLite database and setup data; and the internal modules are Identity, Catalog, Pricing, Purchase, Inventory, Audit and Administration. For Part 4 I moved the container/component design into actual Python files and added a real atomic purchase transaction. The untouched source C4 diagrams are included in the ZIP for traceability.

2. How I used AI to create the code

This is the largest part of the assignment. I used ChatGPT to draft the code, tests and documentation from the existing project specification. I then compared the output with the Call 4 requirements, inspected important logic manually, and ran the tests available in this environment. I did not assume AI output was correct just because it looked complete.

2.1 What I gave the AI

- The 5-page Part 4 assignment and Call 4 implementation discussion, including the unit-test and modularity rules.
- The full Part 3 feature requirements (80 EARS lines), C4 Structurizr DSL and original architecture decision records.
- Known risks from Part 3: inventory overselling, changing rates, money precision, administrator privileges and the audit trail.
- The instruction not to disturb John's earlier work and to package everything for teammate review.

2.2 How the prompts developed

I guided the code generation in smaller, specific steps rather than asking for one large file. The following are summaries of those development instructions, not invented word-for-word logs from a separate prompt-history service.

Iteration	Direction to AI	What it produced / what I checked
1 — Components	Build standalone services that match the C4 components. Keep SQL behind services.	Separate identity, catalog, pricing, inventory, purchase, audit and admin files, plus SQLite schema.
2 — Money and permissions	Only one price function; integer minor units; admin role checked on the server.	Decimal conversion, USD/EUR/JPY handling, saved exchange-rate snapshots, and administrator-only rate updates.
3 — Purchase safety	Atomic ticket count deduction, simulated payment, per-attempt audit, no half-order.	A BEGIN IMMEDIATE purchase transaction, conditional UPDATE, rollback-on-failure and audit insert.
4 — UI and workflows	Connect components in Flask routes and add a clear, responsive ticketing UI.	Registration, login, event search, checkout, history, admin area, single/double-click controls.
5 — Verification	Write tests before claiming correctness; add CI and record the actual run result.	Pytest domain and HTTP tests, a CI workflow, reviewer checklist, and factual test-output summary.

2.3 Was the first code complete and correct?

The initial implementation covered the main modules and pages, but I would not describe it as fully correct on the first draft. Reviewing the components revealed problems around the post-login destination for quick-buy, currency display on event details and the admin ability to alter the USD reference rate. Those were adjusted before the final local test run. Some business rules

cannot be completed correctly without the missing customer emails, and the HTTP tests could not be executed locally because Flask/Werkzeug were unavailable.

2.4 Corrections made during review

Issue noticed	Change made	Why it matters
Quick-buy redirect after login	Signed-out quick-buy now returns to the event detail page, not the POST-only quick-buy route.	After sign-in, the browser can safely GET a page without accidentally purchasing.
Currency mismatch	Event details now compute the preview using a signed-in buyer's saved currency. Checkout can request live server quotes.	Every displayed or charged amount calls the same pricing service.
USD reference currency	Rate administration rejects changing USD away from 1.	Prevents inconsistent base-currency interpretation.
Unknown discount percent	USF-eligible checkouts stop with an explicit pending-configuration message.	Avoids inventing a customer price or silently skipping their expected discount.
False CI confidence	README states tests need dependencies; workflow installs them. Branch protection is a separate maintainer setting.	A passing local syntax check is not the same thing as verified integration tests.

2.5 Implementation approach and component boundaries

The database schema has separate accounts, events, exchange rates, orders and audit tables. Passwords are hashed through Werkzeug, not saved as plaintext. Registration starts every account as a buyer, and administrator roles can only be assigned from a server command. The web route checks the role against the database on every administrator request. Search terms use parameterized SQL and are escaped in page templates.

For purchases, the service first validates the quantity, currency, account and event. It then starts a SQLite IMMEDIATE transaction, computes the exact amount, decrements inventory only if enough tickets remain, writes the order, writes its audit record, and commits everything together. If something fails, it rolls back and creates a separate failure audit. The test for two simultaneous buyers purchasing the final ticket is especially important because the system must not oversell.

The interface is intentionally a small campus-event experience with discoverable event cards, event detail pages, checkout, order history and a private administrator panel. It uses local CSS and JavaScript, not external UI services. The sample events are fictional and labeled as demo data.

2.6 What I would change in the prompts next

With the customer email thread available, I would give the AI the agreed discount percentage, exact scope, cap, hold behavior, rounding policy and confirmation flow as explicit test cases. I would also paste in any failures from GitHub Actions instead of asking for a vague "fix all errors." Each follow-up would identify the smallest affected module, the expected result and a regression test.

3. Testing the application

3.1 What I actually ran

```
python -m compileall -q eventhub tests scripts
python -m pytest -q
python grading/check_part3.py .
```

The final available local run reported: 33 passing tests and 2 skipped test files. The skip was caused by unavailable Flask and Werkzeug dependencies in this offline build environment, not by failing test assertions. The HTTP/authentication tests are supplied in the ZIP but must be run after installing requirements-dev.txt. The earlier Part 3 checker reported 0 failures, with Structurizr CLI unavailable.

3.2 Main unit-test cases

Test area	Cases covered locally
Catalog	Search by artist and venue; case-insensitive matching; no-match fallback; SQL wildcard handling; missing event.
Pricing	USF email case normalization; USD cents, EUR conversion, yen with no minor decimals; invalid rates and quantity; missing/invalid student discount.
Purchases	Successful order and stock decrement; no change on cancel; invalid quantity, currency, out-of-stock and cap errors; rate snapshot after rate change.
Concurrency	Two simultaneous buyer connections compete for one remaining ticket: exactly one order succeeds and stock ends at zero.
Audit and administrator	Exactly one success/failure purchase audit, rate-change audit, rejection of bad rates and changes to the USD reference rate.
Door check-in	A code can be redeemed once, then the second attempt is rejected.
Database	Repeatable seed, nonnegative stock CHECK constraint, newest-first audit records.

3.3 Integration tests included but not yet observed passing

The package contains HTTP tests for login redirects, session expiry, CSRF protection, admin denial, checkout completion, XSS escaping, safe return URLs and quote validation. An identity test file checks password hashing and login failures. In this execution environment those two modules were skipped because required dependencies were not installed. The GitHub Actions workflow is set to install dependencies and run them, but I have not observed a green CI run on the team repository. John should run them in a regular virtual environment before submission.

3.4 Testing limitations

This was not a load test. The two-buyer test is strong evidence for an atomic inventory update but does not establish the unknown peak-load target. Similarly, the eligibility checker validates file form and diagram exports but cannot prove product correctness. I did not claim to verify the website's ability to prevent screenshots; browsers generally do not offer that guarantee.

4. Source code, instructions and submission

All application code and its tests are included in the source ZIP. A teammate can install the requirements, run the test suite and start the app locally. The earlier Part 3 diagrams, requirements and decision files are preserved beside the implementation; no changes were pushed to the shared repository.

4.1 Main files

File or folder	Contents
eventhub/*.py	Eight importable service modules plus Flask web routes
run.py, config/setup.json	Application entry point and fictional event/currency seed data
templates/, static/	Responsive buyer and administrator pages, stylesheet and double-click behavior
tests/	Unit, concurrency and conditional Flask HTTP integration tests
.github/workflows/quality.yml	GitHub Actions checks on push and pull request
features/, docs/decisions/, workspace.dsl, diagrams/	The existing Part 3 source materials, unchanged
docs/ai-process-log.md, docs/implementation-alignment.md	AI work record, limitations and requirement-to-code mapping

4.2 Running it on a normal machine

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
python -m pytest -q
python grading/check_part3.py .
python run.py
```

Open http://127.0.0.1:5000. Sign up with a non-USF demo email for the basic purchase walkthrough. For the USF discount path, only set EVENTHUB_DISCOUNT_PERCENT after the team receives and verifies the actual discount percentage. The demo uses no payment processor and collects no card information.

4.3 What John should review

- Confirm Call 4 email decisions and the policy gaps in docs/implementation-alignment.md before calling the feature set final.

- Install dependencies and obtain zero skipped/failed tests, including HTTP routes and authentication.
- Review all generated code and UI behavior, then check GitHub Actions results and enable branch protections as needed.
- Approve the final narrative and add the actual outcomes of any corrections discovered during review.

My goal was to leave John a working, inspectable Part 4 foundation with the hard parts of the design in code and tests, while keeping our earlier work intact. Where a customer rule was not supplied or verification could not run, I documented it rather than saying everything was perfect.

5. Final review notes

The implementation is meant for the team to review together. It is not a claim that we can launch real ticket sales. Before the final Canvas submission, the team needs to compare the code against the customer-email decisions, run the full environment-dependent test suite, and decide whether any specification lines need to be superseded.

5.1 Short demonstration checklist

- Open the home page on both desktop and mobile; search for an event and a nonmatching name.
- Register a buyer, verify sign-in and sign-out, view an event and simulate one purchase.
- Check the ticket count, history receipt, stored exchange rate and a matching audit entry.
- Try a sold-out purchase, invalid quantity, unsupported currency and missing CSRF token.
- Try two competing buyers for the last ticket; only one should succeed.
- Promote a demo administrator through the server CLI; verify buyer access is refused.
- Change an exchange rate through the admin screen; verify the change is audited and old orders do not change.

5.2 Evidence still needed

Attach or reference a complete CI log after installing Flask and Werkzeug and running the integration suite. Add the missing customer-email decisions to the review record. If the reviewer identifies a problem, keep the failing test and the corrected rerun as evidence of the real AI development and debugging process.